



**FAKULTI TEKNOLOGI DAN KEJURUTERAAN ELEKTRONIK DAN
KEJURUTERAAN KOMPUTER (FTKEK)**

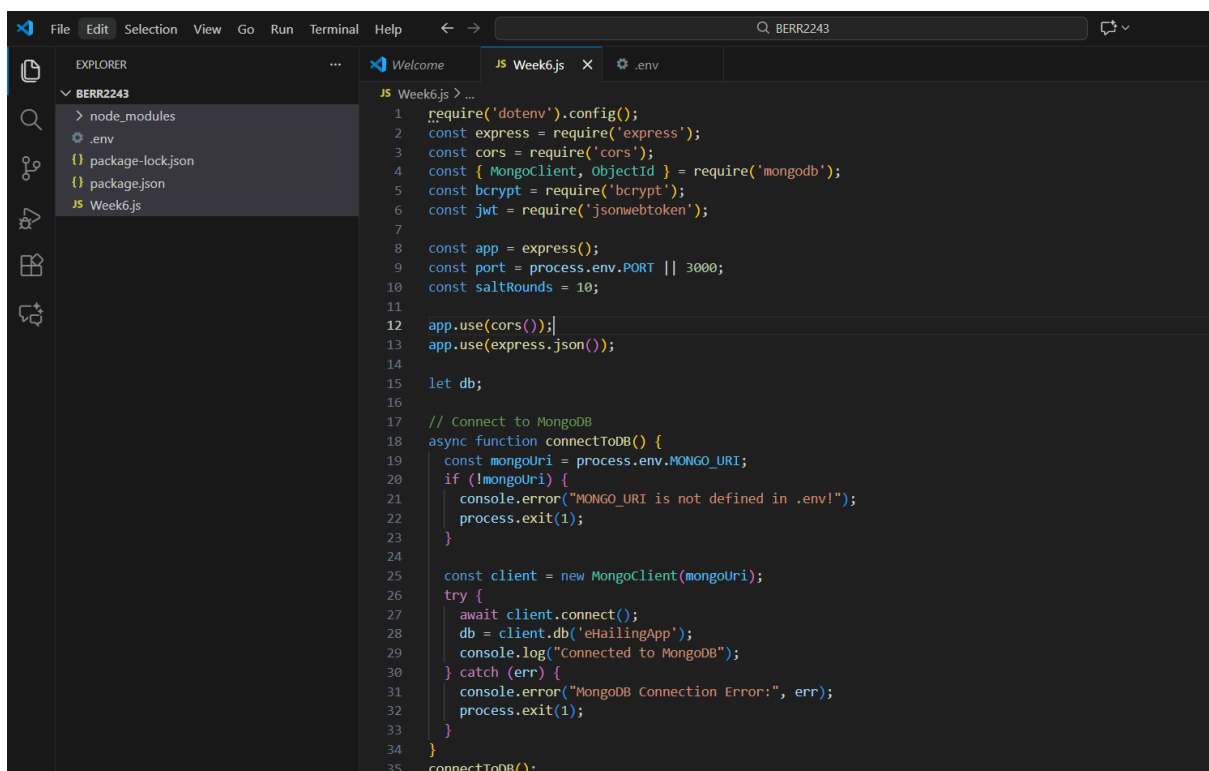
BERR2243 DATABASE AND CLOUD SYSTEM

LECTURER NAME: PM DR SOO YEW GUAN

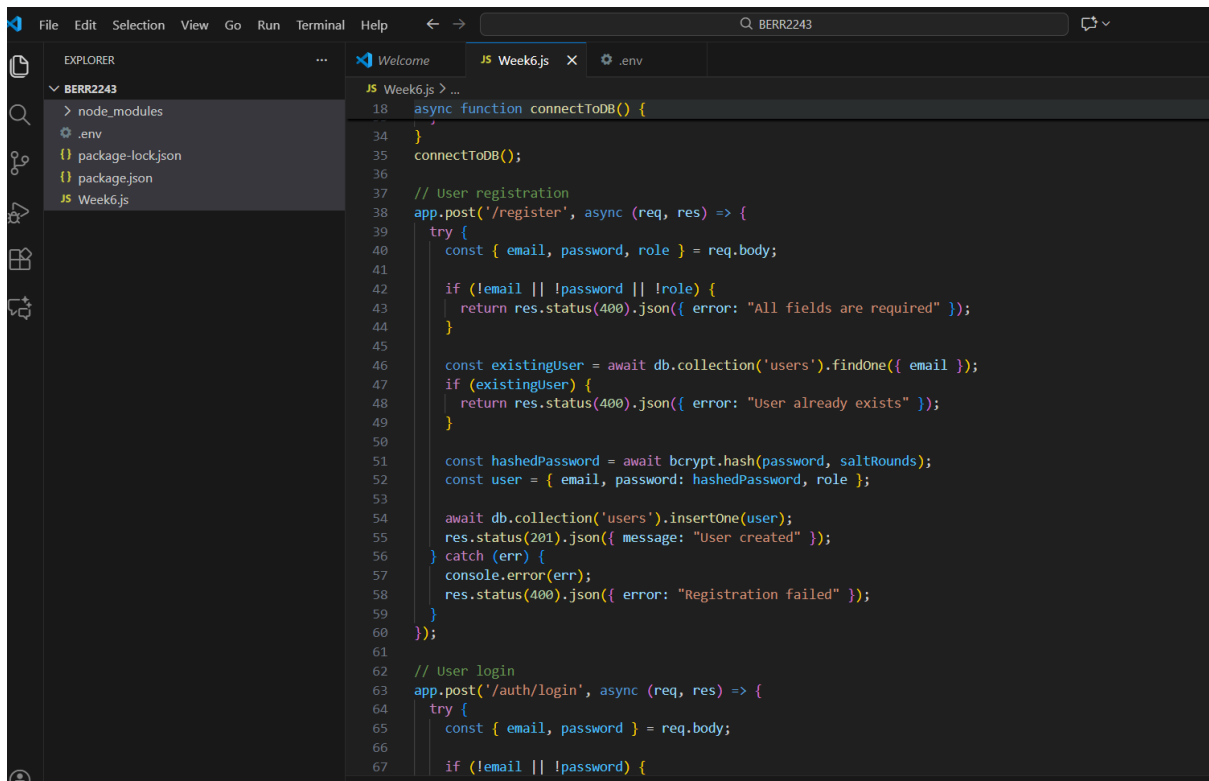
Student Name	No. Matrix	Program
RIGNES A/P TAMIL VANAN	B122320069	BERR/S5
DAMIA NAZURAH BINTI ABDUL RIZAN	B122320020	BERR/S5

Week 6 Lab: Password Hashing, JWT Authentication & Role-Based Access Control

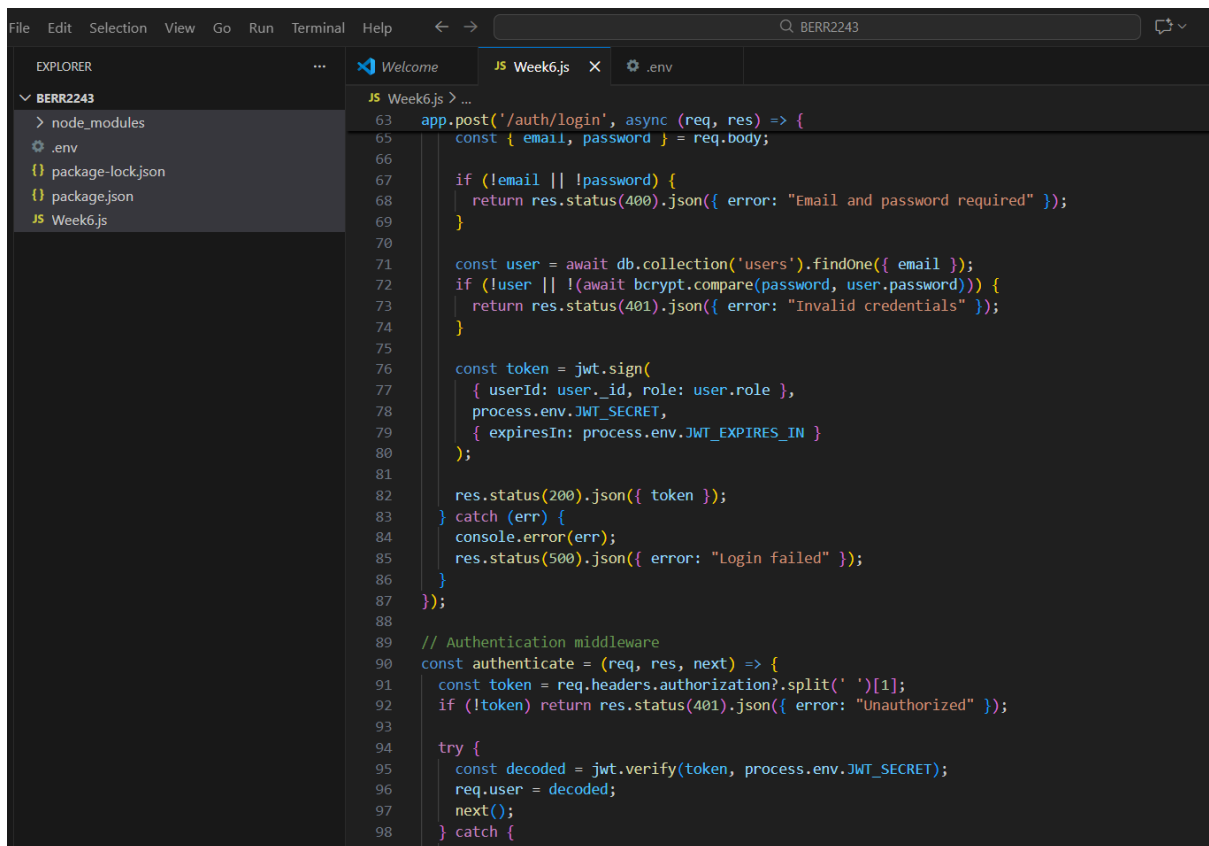
Lab result full coding



```
1 .require('dotenv').config();
2 const express = require('express');
3 const cors = require('cors');
4 const { MongoClient, ObjectId } = require('mongodb');
5 const bcrypt = require('bcrypt');
6 const jwt = require('jsonwebtoken');
7
8 const app = express();
9 const port = process.env.PORT || 3000;
10 const saltRounds = 10;
11
12 app.use(cors());
13 app.use(express.json());
14
15 let db;
16
17 // Connect to MongoDB
18 async function connectToDB() {
19   const mongoUri = process.env.MONGO_URI;
20   if (!mongoUri) {
21     console.error("MONGO_URI is not defined in .env!");
22     process.exit(1);
23   }
24
25   const client = new MongoClient(mongoUri);
26   try {
27     await client.connect();
28     db = client.db('eHailingApp');
29     console.log("Connected to MongoDB");
30   } catch (err) {
31     console.error("MongoDB Connection Error:", err);
32     process.exit(1);
33   }
34 }
35 connectToDB();
```



```
File Edit Selection View Go Run Terminal Help
BERR2243
EXPLORER
  BERR2243
    node_modules
    .env
    package-lock.json
    package.json
    JS Week6.js
JS Week6.js
18 async function connectToDB() {
34 }
35 connectToDB();
36
37 // User registration
38 app.post('/register', async (req, res) => {
39   try {
40     const { email, password, role } = req.body;
41
42     if (!email || !password || !role) {
43       return res.status(400).json({ error: "All fields are required" });
44     }
45
46     const existingUser = await db.collection('users').findOne({ email });
47     if (existingUser) {
48       return res.status(400).json({ error: "User already exists" });
49     }
50
51     const hashedPassword = await bcrypt.hash(password, saltRounds);
52     const user = { email, password: hashedPassword, role };
53
54     await db.collection('users').insertOne(user);
55     res.status(201).json({ message: "User created" });
56   } catch (err) {
57     console.error(err);
58     res.status(400).json({ error: "Registration failed" });
59   }
60 });
61
62 // User login
63 app.post('/auth/login', async (req, res) => {
64   try {
65     const { email, password } = req.body;
66
67     if (!email || !password) {
```



```
File Edit Selection View Go Run Terminal Help
BERR2243
EXPLORER
  BERR2243
    node_modules
    .env
    package-lock.json
    package.json
    JS Week6.js
JS Week6.js
63 app.post('/auth/login', async (req, res) => {
65   const { email, password } = req.body;
66
67   if (!email || !password) {
68     return res.status(400).json({ error: "Email and password required" });
69   }
70
71   const user = await db.collection('users').findOne({ email });
72   if (!user || !(await bcrypt.compare(password, user.password))) {
73     return res.status(401).json({ error: "Invalid credentials" });
74   }
75
76   const token = jwt.sign(
77     { userId: user._id, role: user.role },
78     process.env.JWT_SECRET,
79     { expiresIn: process.env.JWT_EXPIRES_IN }
80   );
81
82   res.status(200).json({ token });
83 } catch (err) {
84   console.error(err);
85   res.status(500).json({ error: "Login failed" });
86 }
87 });
88
89 // Authentication middleware
90 const authenticate = (req, res, next) => {
91   const token = req.headers.authorization?.split(' ')[1];
92   if (!token) return res.status(401).json({ error: "Unauthorized" });
93
94   try {
95     const decoded = jwt.verify(token, process.env.JWT_SECRET);
96     req.user = decoded;
97     next();
98   } catch {
99     return res.status(401).json({ error: "Invalid token" });
100   }
101 }
```

```
JS Week6.js > ...
90 const authenticate = (req, res, next) => {
94   try {
95     const decoded = jwt.verify(token, process.env.JWT_SECRET);
96     req.user = decoded;
97     next();
98   } catch {
99     res.status(401).json({ error: "Invalid token" });
100   }
101 };
102
103 // Authorization middleware
104 const authorize = (roles) => (req, res, next) => {
105   if (!roles.includes(req.user.role)) {
106     return res.status(403).json({ error: "Forbidden" });
107   }
108   next();
109 };
110
111 // Admin delete user
112 app.delete('/admin/users/:id', authenticate, authorize(['admin']), async (req, res) => {
113   try {
114     const userId = req.params.id;
115     const result = await db.collection('users').deleteOne({ _id: new ObjectId(userId) });
116
117     if (result.deletedCount === 0) {
118       return res.status(404).json({ error: "User not found" });
119     }
120
121     res.status(204).send();
122   } catch (err) {
123     console.error(err);
124     res.status(500).json({ error: "Deletion failed" });
125   }
126 });
127
128 // Start server
129 app.listen(port, () => {
130   console.log(`Server running at http://localhost:${port}`);
131 });
```

```
Connected to MongoDB
PS C:\Users\User\Desktop\BERR2243> node Week6.js
Server running at http://localhost:3000
Connected to MongoDB
```

Postman results

Week6 -JWT AUTH / register user

POST http://localhost:3000/register

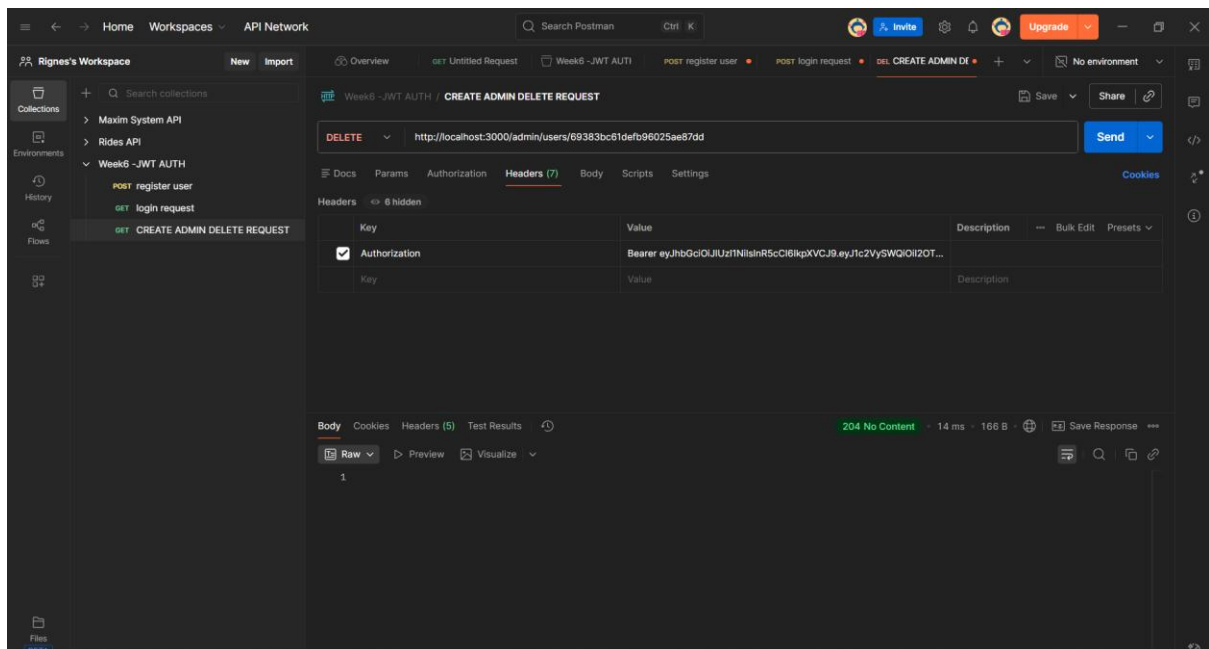
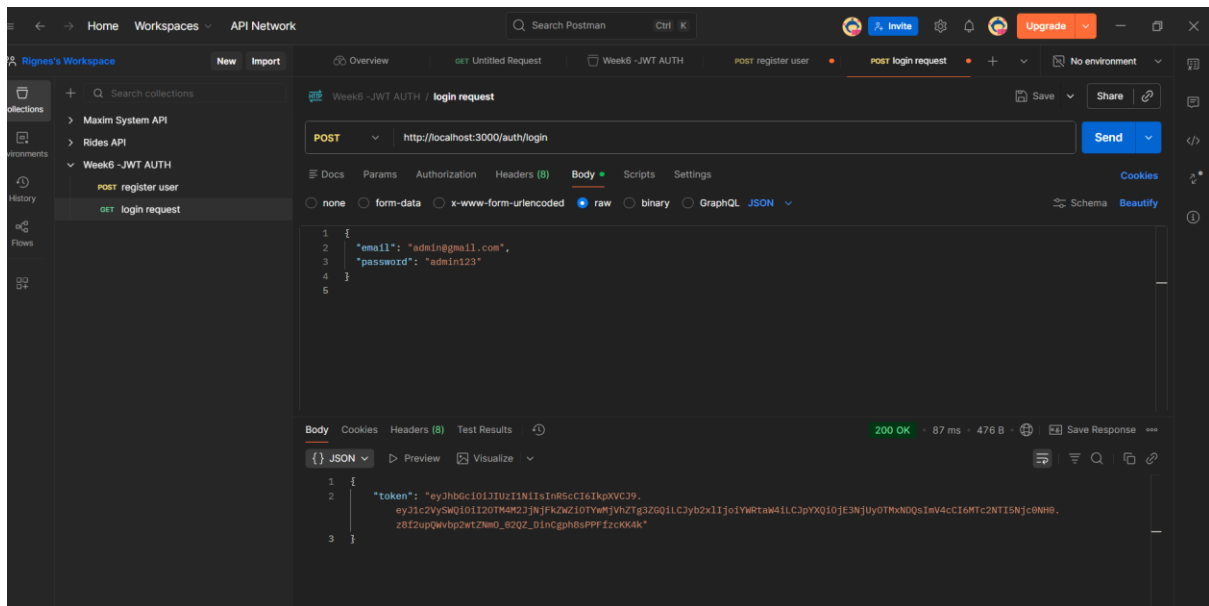
Body

```
{
  "email": "admin@gmail.com",
  "password": "admin123",
  "role": "admin"
}
```

201 Created • 125 ms • 298 B • Save Response

Body

```
{
  "message": "User created"
}
```



Lab Questions

Answer by testing your API in Postman.

1. Token Usage:
 - I. What happens if you omit the Authorization header when accessing `/admin/users/{id}`?
 - It shows 401 Unauthorized.
 - II. What error occurs if you use an expired token?

- It shows again 401 Unauthorized

2. Role Restrictions:

- I. If a customer-role user tries to access `/admin/users/{id}`, what status code is returned?
 - 403 Forbidden
- II. How would you modify the middleware to allow both admin and driver roles to access an endpoint?
 - Use `authorize(['admin', 'driver'])`

3. Security:

- I. Why is the JWT token sent in the Authorization header instead of the request body?
 - JWT in Authorization is secure and stateless.
- II. How does password hashing protect user data in a breach?
 - Hashing ensures plain passwords aren't stored and can't be reversed.

4. Postman Testing:

- I. What is the purpose of the Bearer keyword in the Authorization header?
 - Bearer indicates token-based authentication.
- II. How would you test a scenario where a user enters an incorrect password?
 - expect 401 Unauthorized.